Neural Networks: Multilayered Feed-forward Networks

Video Lesson

The duration is 35:34 minutes.

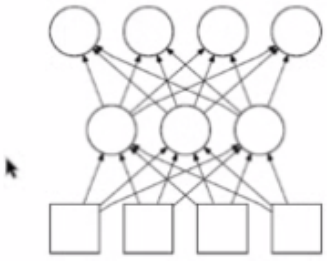
Slide Deck

M12V05 (https://canvas.tamu.edu/courses/430127/files/92570698?wrap=1)

Lesson Transcript

In this lecture, we will look into multilayer neural networks.

Multilayer Feed-Forward Networks

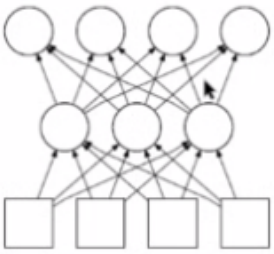


- Proposed in the 1950s
- Proper procedure for training the network came later (1969) and became popular in the 1980s: **back-propagation**

Multilayer feed-forward networks, also known as multilayer perceptrons, were initially proposed in the 1950s. However, initially, the learning rule for these deep connection weights was not available. As for the shallow weight connected to the output, we could use something similar to the perceptron learning rule.

In due time, the proper procedure for training the network was made available in 1969, and this technique became popular in the middle of the 1980s. During this time, it was called back-propagation.

Back-Propagation Learning Rule



- Back-prop is basically a **gradient descent** algorithm.
- The tough problem: output layer has explicit error measure, so finding the error surface is trivial. However, for the hidden layers, how much error each connection eventually cause at the output nodes is hard to determine.
- Backpropagation determines how to distribute the **blame** to each connection.

The back-propagation learning rule is based on gradient descent. Gradient descent depends on

the error surface and how these different parameters affect the error value. This error surface was easy to find for this output layer where the error measure was explicit. We can simply compare the output with the target value in the dataset.

However, these layers in the middle are isolated from the output or the input side, and due to this reason, they are called hidden layers. For these hidden layer neurons, how much error each connection eventually causes at the output was unclear, and how to determine.

Back-propagation determines how to distribute the blame to each connection. For example, if we consider the blame for this particular connection, if this connection contributed some amount of error, then that would end up being computed from these output nodes where the exact error is measured, and those have to be brought back to give the blame for this particular connection.

Gradient Descent

- We want to minimize the total error E by tweaking the network weights.
- E depends on W_i, thus by adjusting W_i, you can reduce E.
- Figuring out how to simultaneously adjust weights W_i for all i at once is practically impossible, so use an iterative approach.
- A sensible way is to reduce E with respect to one weight W_i at a time, proportional to the gradient (or slope) at that point.

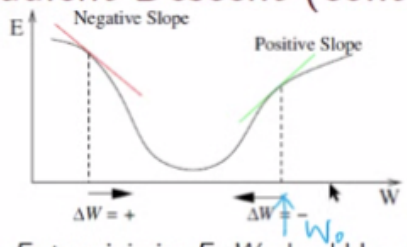
In the general case, in gradient descent, we want to minimize the total error E by tweaking the network weights.

The error function E depends on W, the weights. Thus, by adjusting this weight W_i, we can reduce the error E.

When we have multiple weights, figuring out how to simultaneously adjust all the weights for all index i at once is practically impossible. Hence, we must use an iterative approach to gradually reduce the error E by tweaking the weight values.

One sensible way to reduce the error function E with respect to the particular weight W_i at a time is to make that change proportional to the gradient or the slope at that particular point.

Gradient Descent (cont'd)



$E(w)$

$\left.\frac{dE}{dw}\right|_{w=w_0}$

- For weight W_i and error function E, to minimize E, W_i should be changed according to $W_i \leftarrow W_i + \Delta W_i$:

$$\Delta W_i = \alpha \times \left(-\frac{\partial E}{\partial W_i}\right),$$

where α is the learning rate parameter.

- E can be a function of many weights, thus the partial derivative is used in the above:

$$E(W_1, W_2, ..., W_i, ...W_n, ...)$$

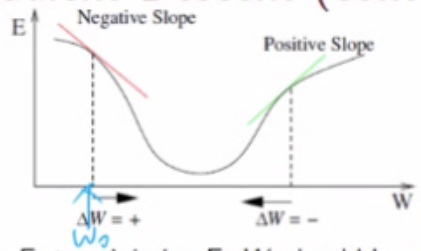
Let us consider this error function, which is a function of one variable *W*, the adjustable value, and the weight. In this case, suppose we can explicitly plot this function and think about the derivative at any point in this range. First, we need to decide which direction to move the weight value.

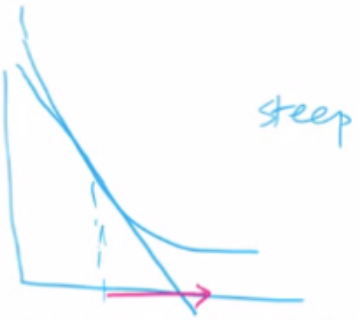
Suppose initially our weight was here.

Then let us say we computed the derivative of this function *E* and insert W_0 into that function, the derivative function. Because here, the slope is positive, it will give us a positive value.

So, if we just use that as the delta weight, then we will end up moving toward the right, and we will end up increasing the error. To go in the opposite direction, we simply need to multiply that derivative with -1.

Gradient Descent (cont'd)





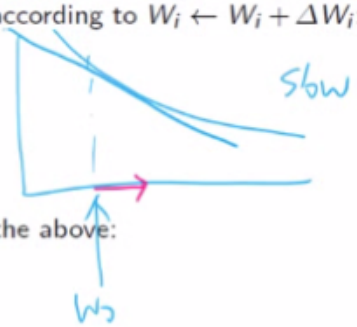
- For weight W_i and error function E, to minimize E, W_i should be changed according to $W_i \leftarrow W_i + \Delta W_i$:

$$\Delta W_i = \alpha \times \left(-\frac{\partial E}{\partial W_i}\right),$$

where α is the learning rate parameter.

- E can be a function of many weights, thus the partial derivative is used in the above:

$$E(W_1, W_2, ..., W_i, ...W_n, ...)$$

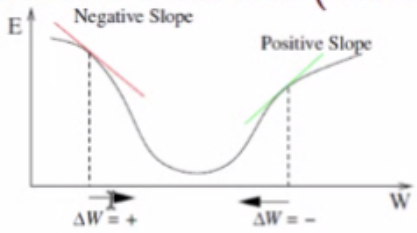


Likewise, when the initial weight value was W_0 here, then the slope will be negative so that the derivative would be negative, but to minimize the energy or the error, we want to move to the right. So we have to multiply it again with -1.

With that, we know how to determine the sign of the change for the ΔW quantity, but what about the amount of change?

The basic idea of gradient descent is to move a lot when this slope is very steep and move just a little bit when the slope is very slow. And that is directly captured by using the derivative here.

Gradient Descent (cont'd)



- For weight W_i and error function E, to minimize E, W_i should be changed according to $W_i \leftarrow W_i + \Delta W_i$:

$$\Delta W_i = \alpha \times \left(-\frac{\partial E}{\partial W_i}\right),$$

where α is the learning rate parameter.

- E can be a function of many weights, thus the partial derivative is used in the above:

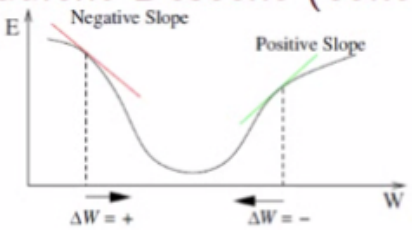
$$E(W_1, W_2, ..., W_i, ...W_n, ...)$$

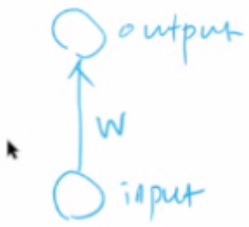
And finally, we have this constant factor α, which is known as the learning rate. So, if this is a large

value, then we will move as much as that in that direction or that direction, depending on the sign. If we have a tiny value, such as 0.1 or 0.01, then we will move, not to the full extent, but just a fraction of that.

So, this α determines how fast we learn.

Gradient Descent (cont'd)





- For weight W_i and error function E, to minimize E, W_i should be changed according to $W_i \leftarrow W_i + \Delta W_i$:

$$\Delta W_i = \alpha \times \left(-\frac{\partial E}{\partial W_i}\right),$$

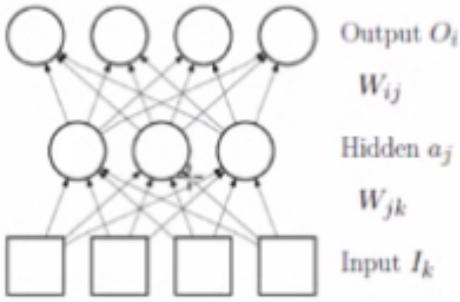
where α is the learning rate parameter.

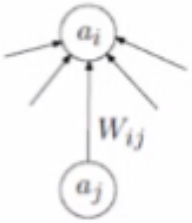
- E can be a function of many weights, thus the partial derivative is used in the above:

$$E(W_1, W_2, ..., W_i, ...W_n, ...)$$

This simple case corresponds to a neural network like this. We have a single input value and then a single output unit. And these are connected with a single weight. But as we know, we generally have many more weight values because we have many more connections. So, this error function *E* is a function of multiple weight variables, and that is why we have this partial derivative instead of the ordinary derivative.

Hidden to Output Weights





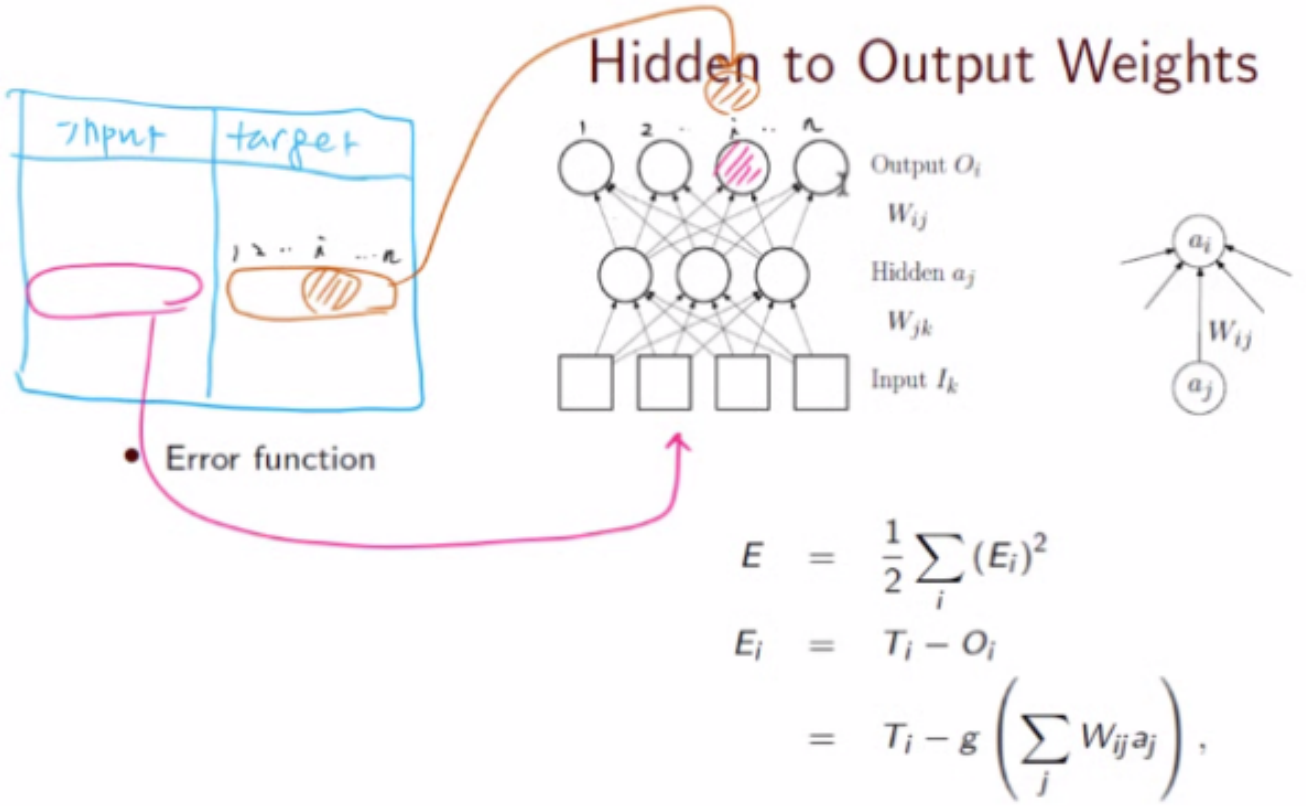
- Error function

$$
\begin{aligned}
E &= \frac{1}{2}\sum_i (E_i)^2 \\
E_i &= T_i - O_i \\
&= T_i - g\left(\sum_j W_{ij} a_j\right),
\end{aligned}
$$

where $g(\cdot)$ is the sigmoid activation function, and T_i the target.

In the following, we will derive the learning rule for these connections from hidden to output and also these deeper connections from the input to the hidden unit. To derive these learning rules, we must first define the error function *E*.

When we have multiple output units like this, then the error is simply 1/2 times the summation of each error at each output unit, squared and summed.

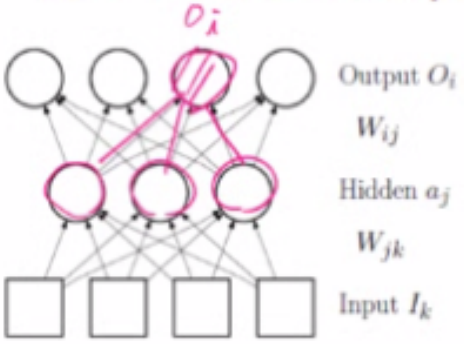


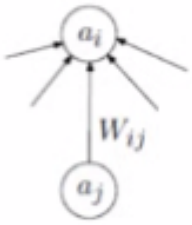
Let us consider this E_i, which is the error in the output unit number i.

E_i is defined as T_i - O_i. T_i is the target value for that unit, and O_i is the output for that unit when given an input. Suppose we want to compute the error E_i from this sample. We take the input part in the dataset and plug it into the neural network, follow through the activation, and finally compute the O_i value.

This O_i value is compared to the T_i value in the target part of the dataset. Because we have multiple output units, we need to have a vector as the target value. So, this i-th entry in this row would now be T_i.

Hidden to Output Weights





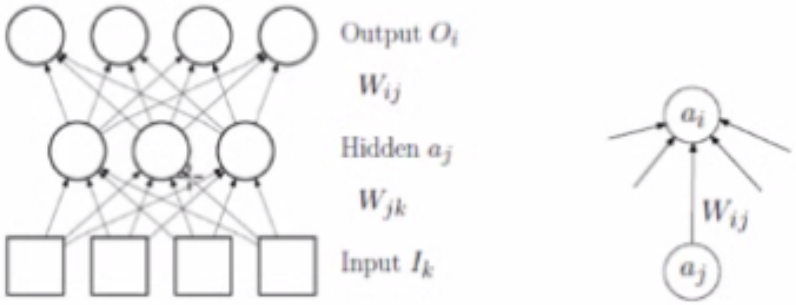
- Error function

$$
\begin{aligned}
E &= \frac{1}{2}\sum_i (E_i)^2 \\
E_i &= T_i - O_i \\
&= T_i - g_I\left(\sum_j W_{ij} a_j\right),
\end{aligned}
$$

where $g(\cdot)$ is the sigmoid activation function, and T_i the target.

O_i here is computed as the weighted sum of the input times the connection weight and passing that through the activation function, g.

Hidden to Output Weights

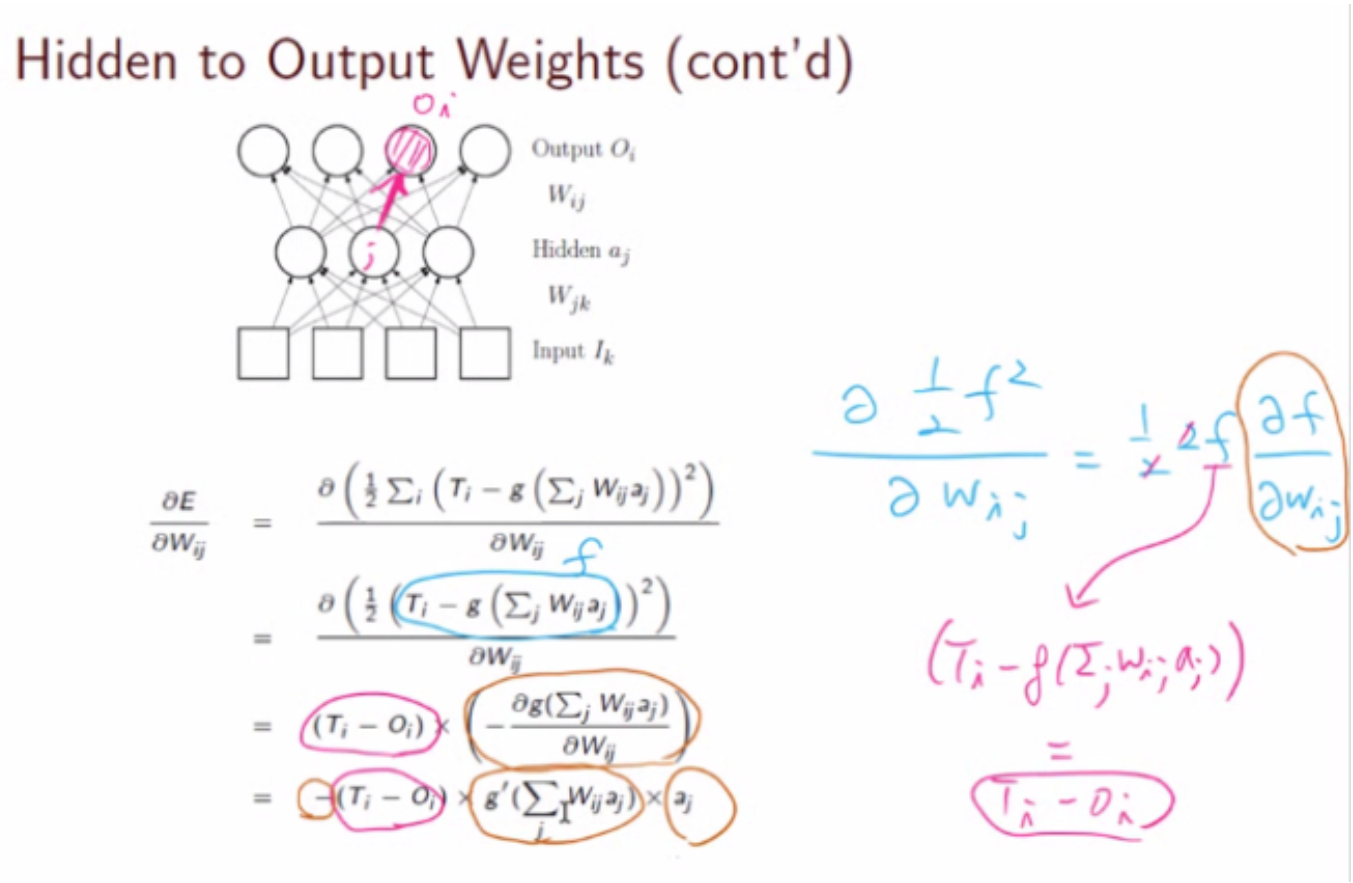


- Error function

$$
\begin{aligned}
E &= \frac{1}{2}\sum_i (E_i)^2 \\
E_i &= T_i - O_i \\
&= T_i - g\left(\sum_j W_{ij} a_j\right),
\end{aligned}
$$

where $g(\cdot)$ is the sigmoid activation function, and T_i the target.

I would like to remind you that whenever we see W_{ij}, this is a connection from *j* to *i,* as shown here, from index *j* to index *i*. So, applying that to this network, when we see W_{ij}, we should assume that the receiving end of that connection is *i*, and then the source would be *j*. Likewise, when we see W_{jk}, then the receiving end has an index of *j*, and the source has an index of *k*.



Now, let us see if we can derive the learning rule for these connections directly attached to one of those output units. So, the output part, the output neuron, is O_i. And when the input neuron is a_j, then this connection would be W_{ij}. What we do is find the partial derivative of the error function with respect to $W_{ij,}$ and using the same definition that we had in the previous slide, just expand E to this full form.

This summation over all i is basically the error squared for each of these, all added up. But this particular connection rate only contributes to the computation of O_i. It is not used in any of these other output units. Hence, when we take the partial derivative with respect to this particular connection weight, then all of these turn out to be constant, so we can get rid of this summation over i and just leave T_i - O_i.

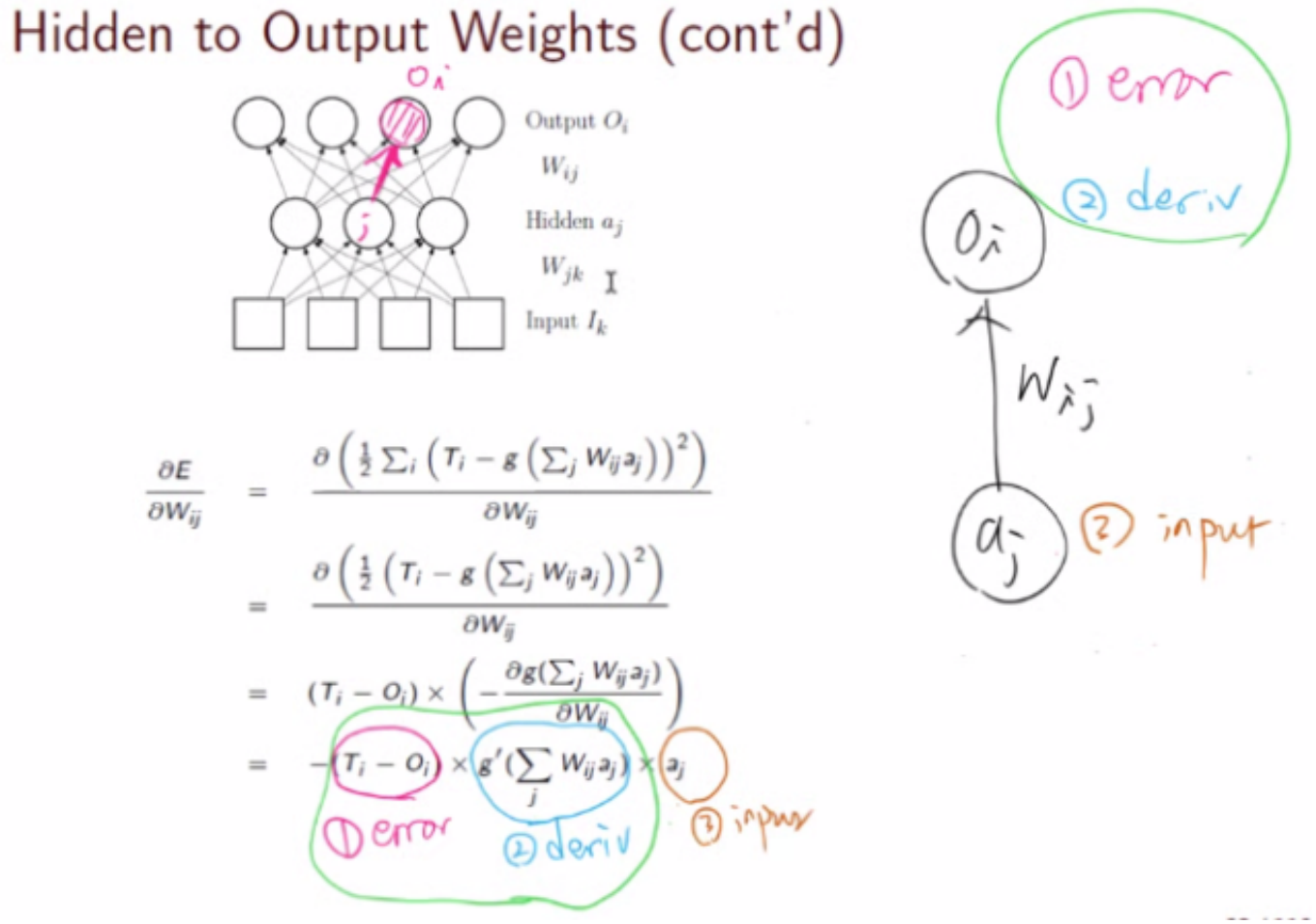
Next, let us set this whole term here, T_i - O_i to f. Then this whole derivative becomes partial of 1/2 f^2 over partial of W_{ij}. When we see this kind of form, we can use the chain rule to get 1/2 × 2 f times the derivative of f with respect to this original variable, W_{ij}.

Because this f was that, we just rewrite it here, but we also realize that g of the sum over j for W_{ij} a_j is simply the definition for the output i. So, we reduce it back to T_i - $O_{i,}$ which becomes that.

When we compute this partial derivative here over f, then T_i is just a numerical value from the dataset. So, this disappears, and we get the partial derivative of that, which is this.

Finally, this derivative becomes minus in the beginning, times the first derivative of this weighted

sum multiplied by the input.



To summarize, what we have here is that the weight will change proportional to that partial derivative value. Since we are multiplying this with -α, minus will cancel out, so the derivative will be used to determine the ΔW value, the weight update value, basically.

This weight update value is now dependent on three quantities. One is the error, which is this, and the second is the derivative of the output side of that connection weight and the input value.

The interesting thing is that these input hidden connection weights will also have the same general structure. So, these connections will be updated based on some error measure of that output, multiplied by its derivative value, multiplied by the input.

Hidden to Output Weights (cont'd)

- For easier calculation later on, we can rewrite:

$$\begin{aligned}\frac{\partial E}{\partial W_{ij}} &= -(T_i - O_i) \times g'(\sum_j W_{ij} a_j) \times a_j \\ &= -a_j \times (T_i - O_i) \times g'(\sum_j W_{ij} a_j) \\ &= -a_j \times \Delta_i\end{aligned}$$

- It is easy to verify $g'(x) = g(x)(1 - g(x))$ from $g(x) = \frac{1}{1+e^{-x}}$, so we can reuse the $g(x)$ value from the feed-forward phase in the feedback weight update:
  - since $g(\sum_j W_{ij} a_j) = O_i$, $g'(\sum_j W_{ij} a_j) = O_i(1 - O_i)$.

To derive the input to hidden weights, it is convenient to define the term delta as the product of the error and the derivative. So, when we have a connection from *j* to *i*, then the derivative is simply the input value times delta on the output side.

Hidden to Output Weights (cont'd)

- For easier calculation later on, we can rewrite:

$$\begin{aligned}\frac{\partial E}{\partial W_{ij}} &= -(T_i - O_i) \times g'(\sum_j W_{ij} a_j) \times a_j \\ &= -a_j \times (T_i - O_i) \times g'(\sum_j W_{ij} a_j) \\ &= -a_j \times \Delta_i\end{aligned}$$

- It is easy to verify $g'(x) = g(x)(1 - g(x))$ from $g(x) = \frac{1}{1+e^{-x}}$, so we can reuse the $g(x)$ value from the feed-forward phase in the feedback weight update:
  - since $g(\sum_j W_{ij} a_j) = O_i$, $g'(\sum_j W_{ij} a_j) = O_i(1 - O_i)$.

Another quick trick is to replace this derivative with output *i* times 1 minus output *i*, which comes directly from the property of the sigmoid function. When *g*(*x*) is the sigmoid function, then the derivative of *g*(*x*) is *g*(*x*) times 1 minus *g*(*x*).

Hidden to Output Weight Update

- From $\frac{\partial E}{\partial W_{ij}} = -a_j \times \Delta_i$, we get the update rule:

$$W_{ij} \leftarrow W_{ij} + \alpha \times a_j \times \Delta_i,$$

where $\Delta_i = (T_i - O_i) \times g'(\sum_j W_{ij} a_j)$.

To summarize, the hidden to output weight update from this partial derivative, which is now minus $a_j \times \Delta_i$, Δ_i being the product of the error and the derivative. Then when we multiply -α by this, we get the weight update rule like this: W_{ij} is updated as the previous W_{ij} value plus α, the learning rate, times $a_j \times \Delta_i$.

Input to Hidden Weights

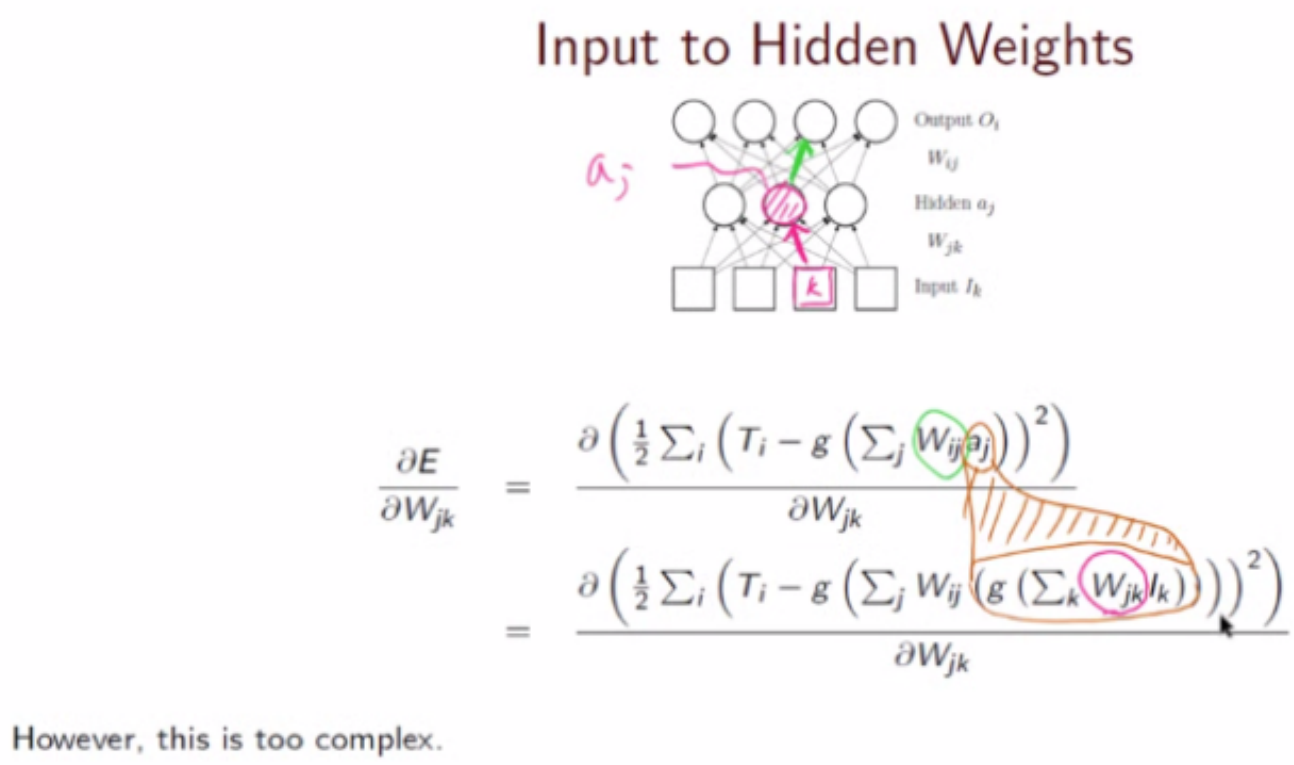
$$\frac{\partial E}{\partial W_{jk}} = \frac{\partial \left(\frac{1}{2} \sum_i \left(T_i - g \left(\sum_j W_{ij} a_j \right) \right)^2 \right)}{\partial W_{jk}}$$

$$= \frac{\partial \left(\frac{1}{2} \sum_i \left(T_i - g \left(\sum_j W_{ij} \left(g \left(\sum_k W_{jk} I_k \right) \right) \right) \right)^2 \right)}{\partial W_{jk}}$$

However, this is too complex.

Now let us consider updating this connection weight.

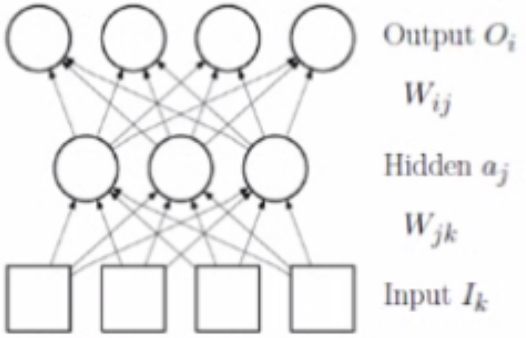
What we can do is first just write down the partial derivative as before, where we expand this error function to 1/2 summation over all *i* on the output times T_i minus the output of *i* squared. So, that is the error squared in each of these output units.



The problem is, in this error function, we do not see W_{jk}. The only way that we can see is W_{ij}, which is this connection when we have these connections.

To make this connection W_{jk} to be visible in this equation, we need to expand a_j to this, which is the weighted sum of the input. And these connection weights pass through the activation function. Finally, we see W_{jk} in this error equation. But directly computing this partial derivative in this form is very difficult.

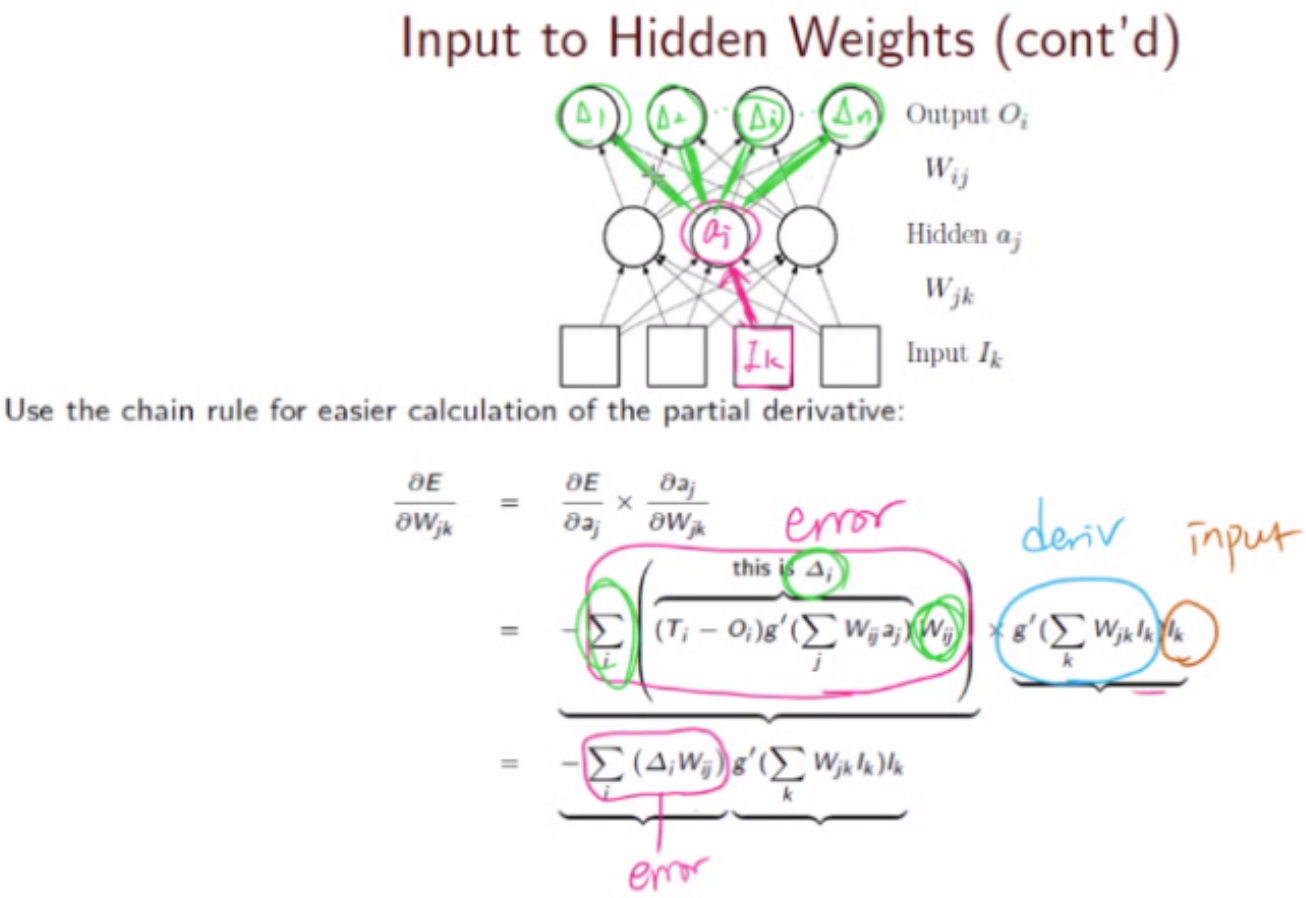
Input to Hidden Weights (cont'd)



Use the chain rule for easier calculation of the partial derivative:

$$\begin{aligned}
\frac{\partial E}{\partial W_{jk}} &= \frac{\partial E}{\partial a_j} \times \frac{\partial a_j}{\partial W_{jk}} \\
&= \underbrace{-\sum_i \left(\overbrace{(T_i - O_i) g'(\sum_j W_{ij} a_j)}^{\text{this is } \Delta_i} W_{ij} \right)}_{} \times \underbrace{g'(\sum_k W_{jk} I_k) I_k}_{} \\
&= \underbrace{-\sum_i (\Delta_i W_{ij})}_{} \underbrace{g'(\sum_k W_{jk} I_k) I_k}_{}
\end{aligned}$$

The trick here is to use the chain rule. Instead of trying to compute this directly, we transform this into the partial derivative of E over a_j, multiplied by the partial derivative of a_j over W_{jk}.

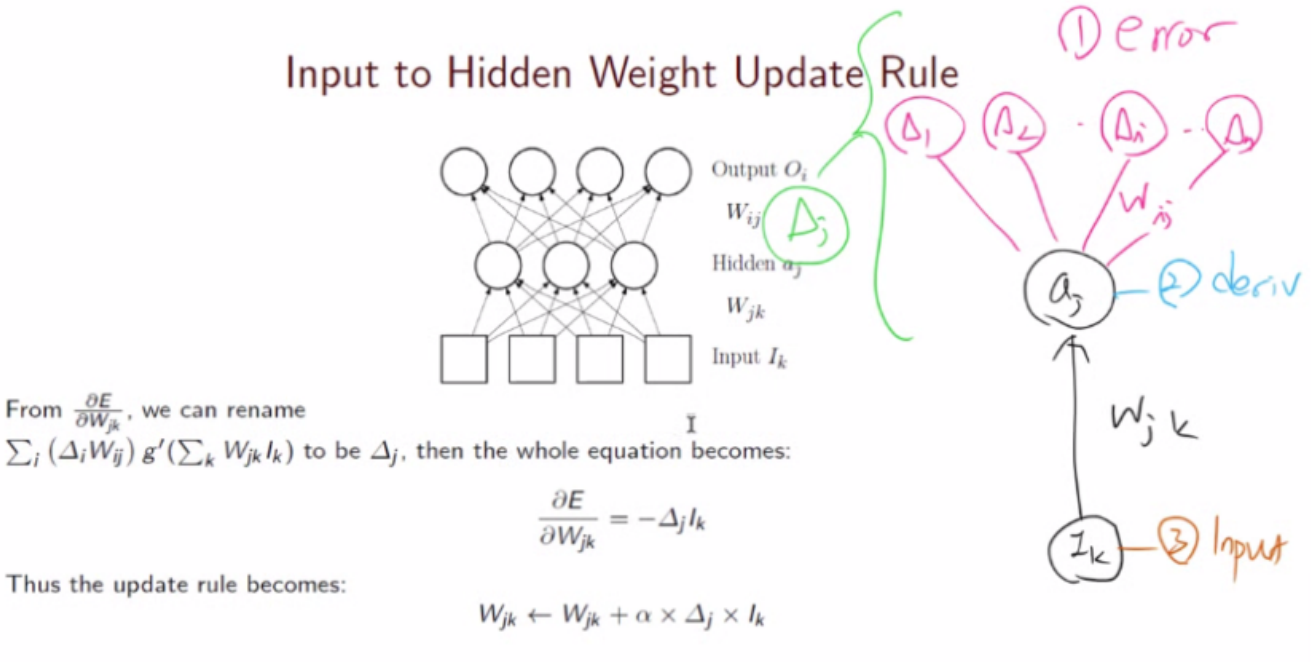


This full derivation takes about two slides, and we will not go through that and just skip to the final result. So, when we use this chain rule, then what happens is that this first part will become this, which will basically determine the error, and the second part here will become the derivative times the input.

Again, we end up with the same error times derivative times input structure as we have seen in the hidden to output connection weight update. So, these two are the same as before, whereas this is the only different part.

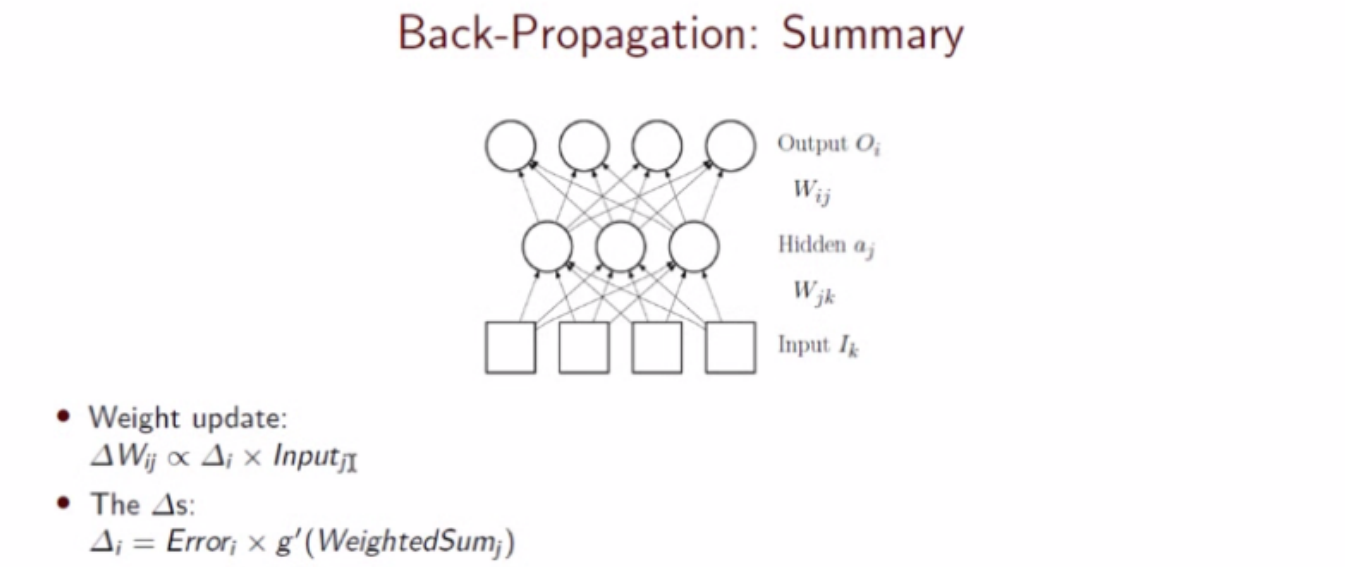
Let us take a look at this error term more closely. Here we have a summation over all output units, and then we have T_i - O_i times the first derivative of the weighted sum and W_{ij}, which would correspond to these connection weights. Conveniently, this product here is Δ_i, as we defined earlier. So, this reduces to $\Delta_i \times W_{ij}$.

So, if we computed all these Δ_i values here, then we can reuse that by multiplying the Δ_i with the connection weight and simply summing them up. So, the error for this unit, a_j, is the back-propagated delta values from above through these connection weights.



In the same manner that we define Δ_1, Δ_2, Δ_i as a product of the error times derivative for the output unit, we can also define Δ_j for this unit as a product of this error, the back-propagated delta, times the derivative as shown here. Then the whole learning equation, which will be determined by this partial derivative, is $-\Delta_j \times I_k$.

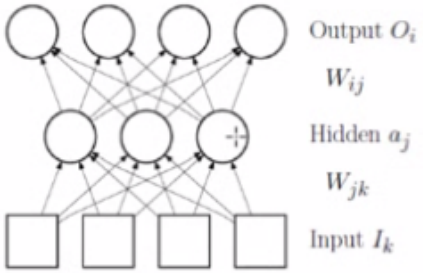
When we multiply -α by that, then we get the updated quantity $\alpha \times \Delta_j \times I_k$.



In summary, wherever our connection weight is, whether it is at the output unit or anywhere in the depths of the neural network, the change in weight, $W_{ij,}$ is proportional to Δ_i of the output side of the connection, multiplied by the input value of *j*. In contrast, these delta values are defined as

errors in *i* times the derivative.

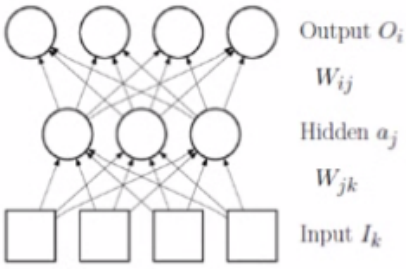
General Case: More Than 2 Layers

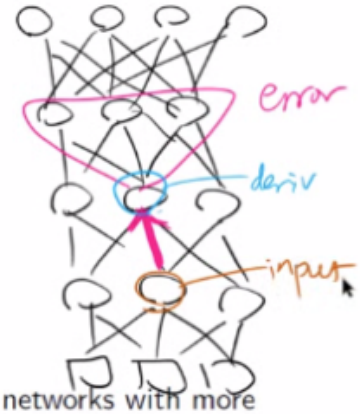


- In general, the same rule for back-propagating Δs apply for multiple layer networks with more than two layers.
- That is, Δ for a deep hidden unit can be determined by the product of the weighted sum of feedback Δs and the first derivative of feedforward weighted sum at the current unit.

As I alluded to in the previous slides, when we have multiple layers, more than two layers, as shown here, we can still use the same rule.

General Case: More Than 2 Layers





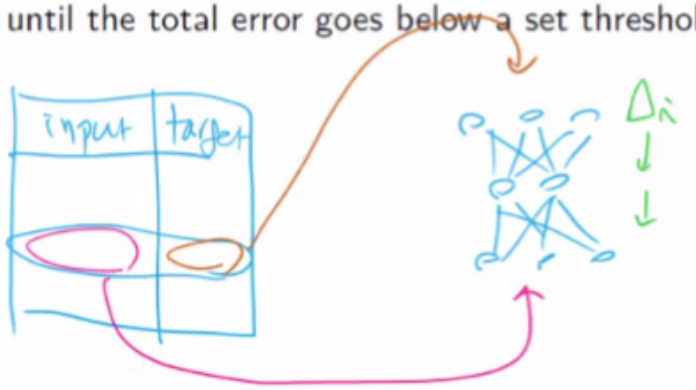
- In general, the same rule for back-propagating Δs apply for multiple layer networks with more than two layers.
- That is, Δ for a deep hidden unit can be determined by the product of the weighted sum of feedback Δs and the first derivative of feedforward weighted sum at the current unit.

For example, we have 1, 2, 3, 4-layer neural networks. To update this connection weight here, we only need information from right above, the delta values, and the connection weights from which we compute the error, compute the derivative of the output side of that connection weight, and then look up the input value. And this is all we need.

So, learning is local. We do not need any further information from above or deeper below.

Backpropagation Algorithm

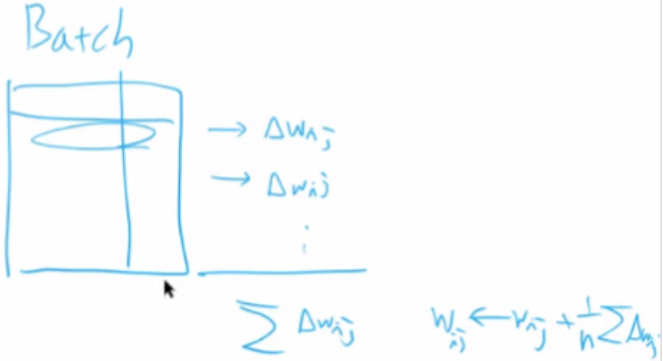
1. Pick (*Input*, *Target*) pair.
2. Using input, activate hidden and output layers through feed-forward activation.
3. At the output node, calculate the error $(T_i - O_i)$, and from that calculate the Δs.
4. Update weights to the output layer, and backpropagate the Δs.
5. Successively update hidden layer weights until Input layer has been reached.
6. Repeat step1–5 until the total error goes below a set threshold.



Here is a back-propagation algorithm. We pick the input and target pair from our dataset. Using this input, plug it into the neural network. Compute the output. At the output node, we compute the error by comparing the target value and the output generated from which we derived the delta values. Using these, we can update these connection weights at the output layer, and we can back-propagate these delta values to compute the error for these hidden units and successively update the hidden layer weights until we reach the input layer. And we repeat this step. Pick another input and target value, update the weights, and so on.

Technical Issues in Training

- Batch vs. online training
  - Batch: accumulate weight updates for one epoch, and then update
  - Online: immediately apply weight updates after one input-output pair.
- When to stop training
  - Training set: use for training
  - Validation set: determine when to stop
  - Test set: use for testing performance



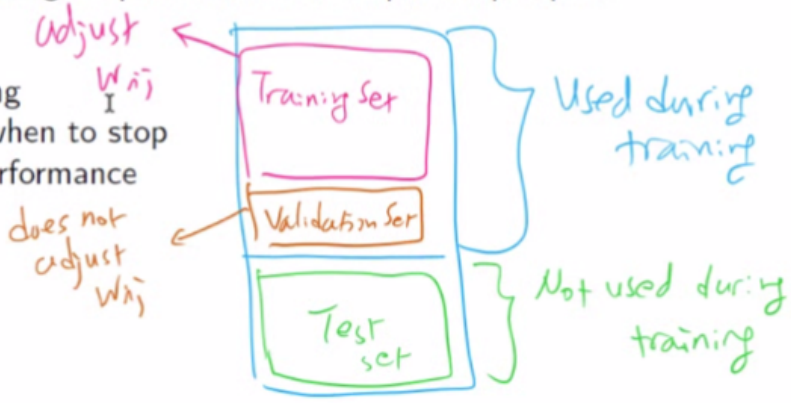
There are several ways of training our neural network. First is a batch training mode. In this case, we accumulate the weight updates for each epoch and then update. Epoch means the sweep across the entire dataset once. For each sample, we compute a delta weight, and instead of applying that to the weight, we keep on adding those up, and in the end, once we have done the complete sweep, there is one epoch.

Then we apply this average update to the weight. So, that is batch training mode.

In online training, we immediately apply the weight updates after one input-output pair. As shown here, we take one input and output, the target value pair, compute the W_{ij} change, that is, the ΔW_{ij}, and immediately apply that to that weight. Next, we take the next sample, compute the ΔW_{ij}, and again, immediately apply that.

Technical Issues in Training

- Batch vs. online training
  - Batch: accumulate weight updates for one epoch, and then update
  - Online: immediately apply weight updates after one input-output pair.
- When to stop training
  - Training set: use for training
  - Validation set: determine when to stop
  - Test set: use for testing performance



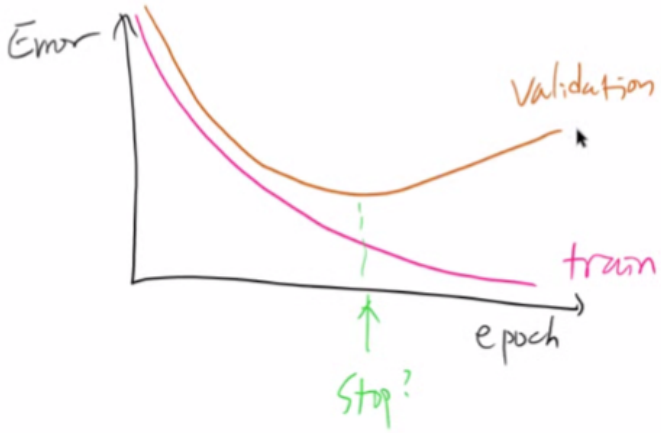
Typically, a data set is divided into three parts. The training set, the validation set, and the test set. Both the training set and validation set are used during training. However, only the training set is used to adjust the connection weight values, whereas the validation set is only used to measure the performance of the training set.

The validation set samples are not used to adjust the connection weights, W_{ij}. In comparison, the test set is not used during the training in any way, either as the set that is used to adjust the connection weights or those that are used to measure the progress.

One common use of this validation set is to determine when to stop the training.

Technical Issues in Training

- Batch vs. online training
  - Batch: accumulate weight updates for one epoch, and then update
  - Online: immediately apply weight updates after one input-output pair.
- When to stop training
  - Training set: use for training
  - Validation set: determine when to stop
  - Test set: use for testing performance



What typically happens during training is that the error will continuously decrease for the training set samples.

However, because we are not using the validation set samples to adjust the weights, initially, it will go down, whereas, after a certain while, this validation error will start to increase. This is an indication that our model is overfitting. So, the typical strategy here is to stop the training when our validation error increases.

The rationale is that this validation set, again, not being used to adjust the connection weights, would probably be similar to the test set performance.

Problems With Backprop

- Learning can be extremely slow: introduce momentum, etc.
- Network can be stuck in local minima: this is a common problem for any gradient-based method.

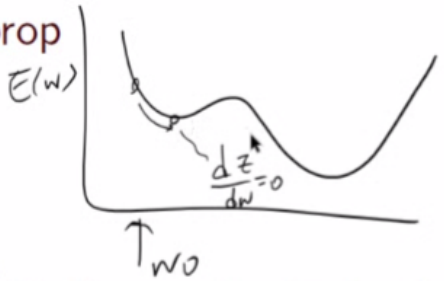
Other issues are: how to introduce new batches of data after the training has been completed.

There are several problems with back-propagation. First, the learning can be very slow, which is

typical of any gradient descent-based approach. To solve this issue, we can introduce something called momentum and other approaches.

This is a more serious problem: the network can get stuck in local minima.

Problems With Backprop



- Learning can be extremely slow: introduce momentum, etc.
- Network can be stuck in local minima: this is a common problem for any gradient-based method.

Other issues are: how to introduce new batches of data after the training has been completed.

Consider this error function is shaped like this, and suppose our initial weight value was here. Then following this gradient direction, we will end up in this location, which is a local minimum, and as we can see, the derivative at that point would be zero. Because our delta weight is proportional to this derivative value, what happens is that the weight update will stop once we reach this minimum here. As we can see, that is not the global minimum. It is only a local minimum. So, if we start anywhere within this range, if we start here, then we will go left and get stuck. In over half of this range, we will get stuck in the local minimum if we initialize the initial weight to be again within that range.

Problems With Backprop

- Learning can be extremely slow: introduce momentum, etc.
- Network can be stuck in local minima: this is a common problem for any gradient-based method.

Other issues are: how to introduce new batches of data after the training has been completed.

Another issue is how to introduce new batches of data after training has been completed.

Problems With Backprop

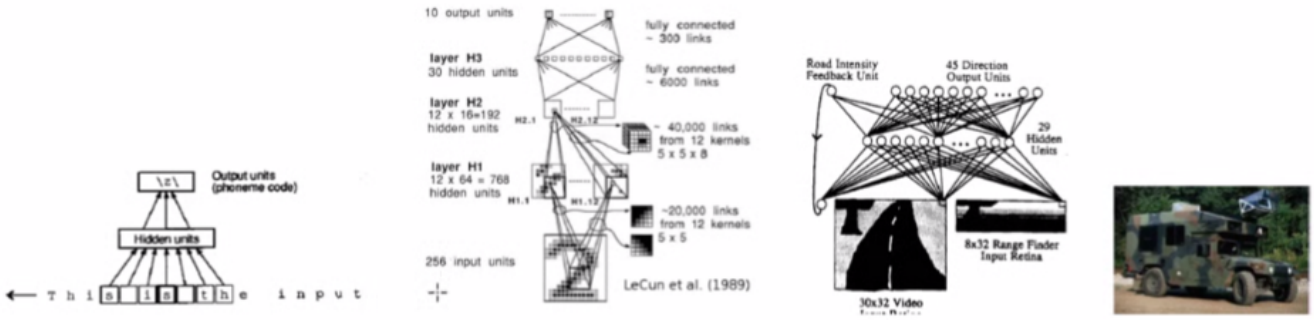
- Learning can be extremely slow: introduce momentum, etc.
- Network can be stuck in local minima: this is a common problem for any gradient-based method.

Other issues are: how to introduce new batches of data after the training has been completed.

Catastrophic forgetting

This is the problem of catastrophic forgetting when we use one data set to train and then, later on, take a new dataset and only train that model with just that new dataset. Then it will forget about the first dataset.

Backprop Application



- Speech generation: NetTALK (Sejnowski and Rosenberg, 1987)
- Character recognition: LeCun (1989)
- Driving a car: ALVINN (1993), etc.
- Since 2012, a flood of deep neural networks applications based on backprop learning: vision, NLP, protein folding, robot control, autonomous driving, etc.

Multi-layer neural networks trained using back-propagation emerged in the middle of the eighties. And it quickly found interesting applications—for example, speech generation. Pose we type in a sentence like this; this is the input. Then we want to generate the corresponding speech, and the neural network, which learned to do this, is very simple.

It is just a simple 2-layer neural network. And the input would be these characters in this seven-character window. Given this context, we want to map this central character to its proper phoneme. Because this says, "this is the input," this *s* character would now map to this \z\ phoneme.

If this window was centered around this character, *s*, then the corresponding phoneme would be \s\ because this ends with the sound \s\.

And this neural network was called NetTALK, which came out in 1987. And we will listen to some of these results, which are very entertaining.

Next, around 1989, Yann LeCun, one of the Turing Award winners recently for his contributions to deep learning, came up with this very popular architecture called Convolutional Neural Network.

So, we plug in an image, a handwritten digit, and it goes through multiple convolution layers to end up in these 10 output units that correspond to the 10 digits, zero to nine.

We will look at this convolutional neural network in more detail when we discuss deep learning in the next module.

These multilayer neural networks have also been applied to autonomous driving. This was work done in 1993, and then this system is called ALVINN. We can see this huge military truck, which has all the equipment in it, but underlying, that was a very simple neural network like this. Just a two-layer neural network, which takes in the video input and range finder input. Using that, it will generate a steering direction using these 45-direction output units. To collect data, someone would actually drive this vehicle while recording the video and also recording the steering movements, and we use the same back-propagation learning algorithm to train these connection weights, and they were able to drive this vehicle autonomously without any human intervention on a highway for over a hundred miles.

Since 2012, there has been a flood of deep neural network applications. These are based on back-propagation learning, such as vision, natural language processing, protein folding robot control, autonomous driving, etc.

Demo: NetTALK

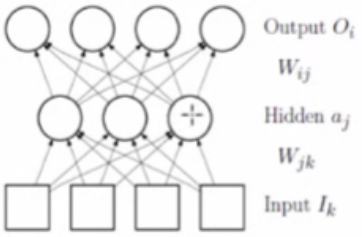
- I want to I want to go to my grandmother's ....
- friend, sent, around, not, red, soon, doubt, key, attention, lost

Just for fun, let us listen to some of these demo sound files from the NetTALK system.

This audio clip is from the beginning when the weights are not known.

"... ... ..." Well, that is very interesting.

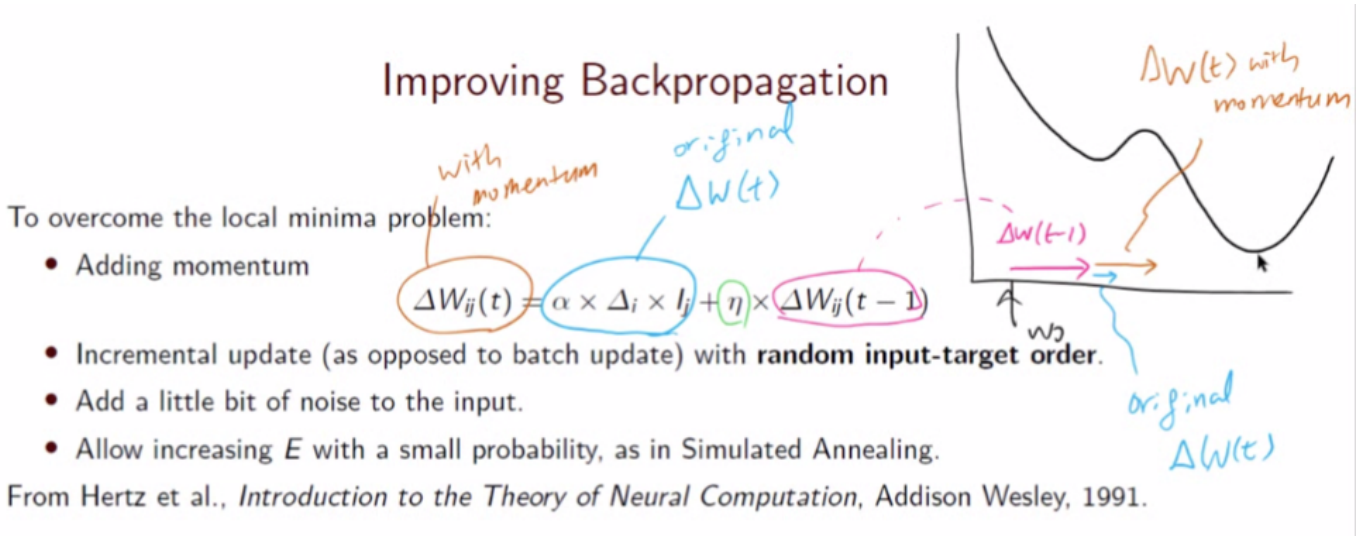
Another Application of Backpropagation: Image Compression



- Image compression
  1. target output is the same as the input (autoencoder).
  2. hidden layer units are fewer than the output (and input) layer units.
  3. the hidden layer forms the compressed representation.

Another application of back-propagation is image compression or file compression. In general, the idea is very simple. Suppose we have an image; then we plug in the image as the input. As the target output, we plug in the same input image, and this kind of arrangement is generally called the autoencoder. The key here is that the hidden layer has fewer units than the input layer. So, when we take this part, and compute the hidden unit activations from an image, then that is a compressed representation.

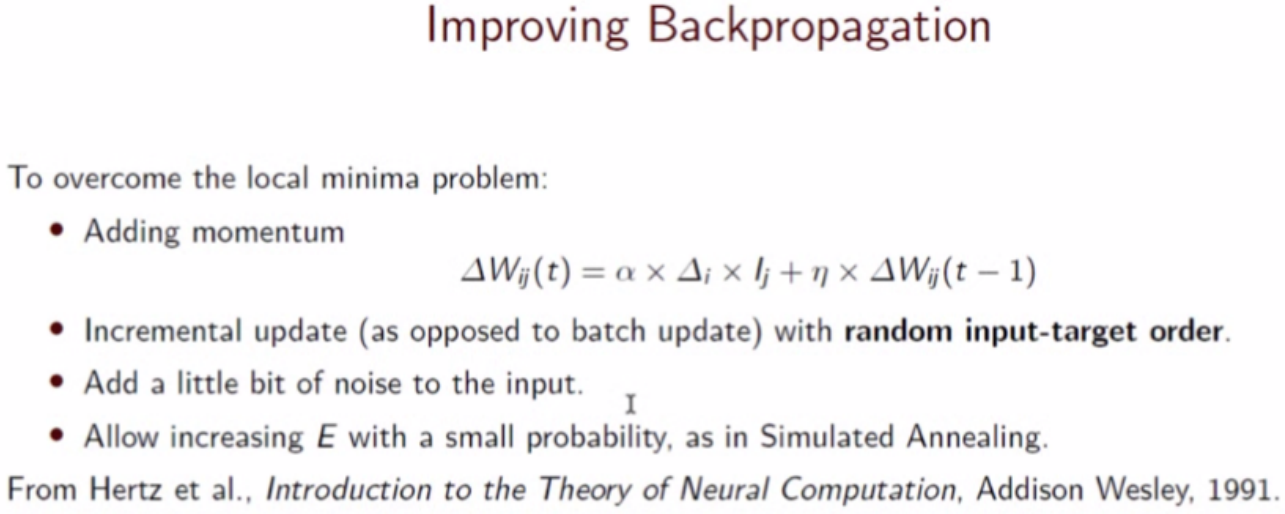
Once we have this compressed representation, we can take it to the second part of the learned neural network. When we plug in the compressed hidden representation, then it will generate output, which is supposed to be very similar to the original input.



One way to improve back-propagation and also to overcome the local minima problem is to add something called momentum. So, here is the basic idea. Initially, we start at W_0, and let us say this was the weight update amount at times step t - 1. As we can see, the slope is very slow at this location, so $\Delta W(t)$ would be very small.

So, if we just use this original $\Delta W(t)$, we get stuck in this local minimum. This corresponds to the original $\Delta W(t)$. However, if we add a fraction of the previous ΔW, which is $\Delta W(t - 1)$, then we will end up with this resulting weight update.

Then we will be able to go over this hump and continue on to the global minimum.



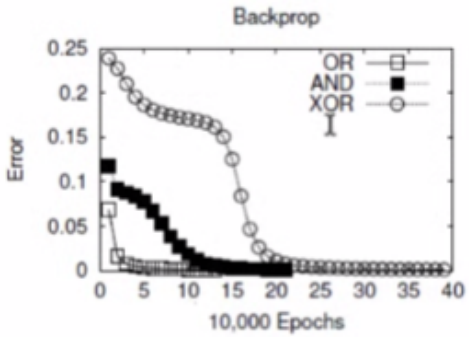
Another approach to avoid the local minimum is to use incremental updates with random input-target order. Here we are not using batch training but online training where we can add a little bit of noise to the input or use something like a simulated annealing strategy by allowing increasing E with a small probability.

Backpropagation Exercise

- **URL:** `https://github.com/tamu-edu/clen-csce-choe-backprop/`
- Click on [Use this template] and create a new repo.
- Read the `README` file:
- Run `make` to build (on unix).
- Run `./bp conf/xor.conf` etc.

If you are interested in writing the back-propagation algorithm from scratch, you can take a look at my GitHub repo shown here. You can use the template to create a new repo and read the README file. This is all written in C++, and you can run a particular dataset, for example, the XOR dataset using this back-propagation command.

Backpropagation: Example Results

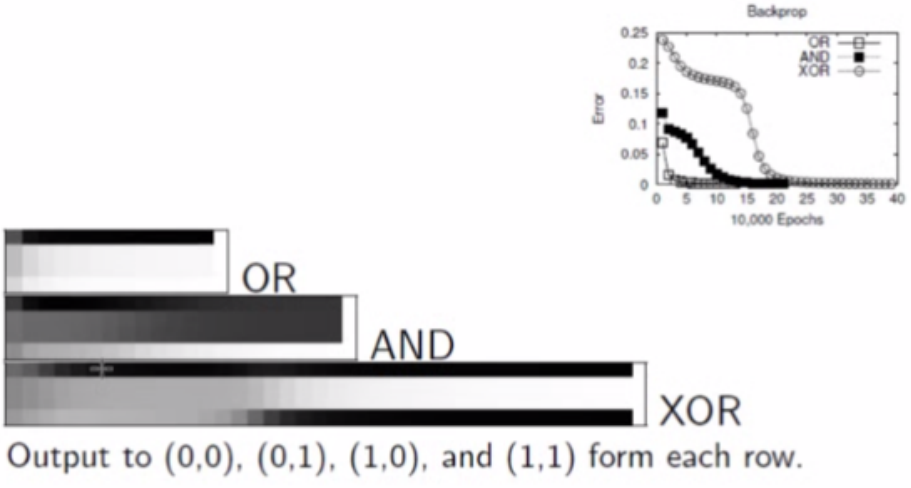


- Epoch: one full cycle of training through all training input patterns.
- `OR` was easiest, `AND` the next, and `XOR` was the most difficult to learn.
- Network had 2 input, 2 hidden and 1 output unit. Learning rate was 0.001.

Here are some example results. So, the Y-axis is the error, and the X-axis is the epochs. Here I trained OR and then XOR. As we can see, OR and AND converge to a near-zero error quickly.

Whereas XOR takes a little bit longer.

Backpropagation: Example Results (cont'd)



We can also look at the output for the four different inputs: (0, 0), (0, 1), (1, 0), (1, 1), and for OR, it should be black, white, white, white. It quickly shows that particular desired output. For AND, it should be black, black, black, white. It takes a while before it gets there. There is an error in the formatting; the last column should be ignored.

For XOR, it should be 0, 1, 1, 0, so it should be black, white, white, and black. It only gets there around this point.

Backpropagation: Things to Try

- How does increasing the number of hidden layer units affect the (1) time and the (2) number of epochs of training?
- How does increasing or decreasing the learning rate affect the rate of convergence?
- How does changing the slope of the sigmoid affect the rate of convergence?
- Different problem domains: handwriting recognition, etc.

There are several things to try.

How does increasing the number of hidden layer units affect the time and the number of epochs

for training?

And how does increasing or decreasing the learning rate affect the convergence rate?

And how does changing the slope of the sigmoid affect the rate of convergence?

And what happens if we apply it to a different domain like handwriting recognition and so on?

That is it for multilayer networks. In the next lecture, we will look at unsupervised learning.